

**QORFINDER: AN EFFICIENT SEMANTIC SEARCH DESKTOP APP FOR
LOCAL FILES USING NATURAL LANGUAGE PROCESSING**

By

Khor Yu Zhuang

A PROPOSAL

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Faculty of Information and Communication Technology

(Kampar Campus)

FEBRUARY 2026

ABSTRACT

With the explosion of digital documents, local file systems have become a fragmented mess. Conventional lexical search approaches compound this problem by relying on exact matches of keywords and do not align with the conceptual nature of human memory recall. Though Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) address this semantic search problem, they require users to either sacrifice privacy by using cloud-based services, or to use costly dedicated hardware for local computing. Our project introduces QorFinder, a desktop-based, privacy-conserving, and efficient semantic file search system that is exclusively for Windows 10/11.

QorFinder is engineered to bypass the computational bloat of generative AI by focusing strictly on optimized semantic retrieval. Utilizing an Iterative and Incremental Development (IID) methodology, the system architecture integrates a memory-safe Rust backend, the lightweight Tauri framework for the graphical user interface, and Qdrant as an embedded vector database. By executing highly compressed sentence transformer models directly on the local CPU via the ONNX runtime, QorFinder generates and queries dense vector embeddings offline. This approach guarantees absolute data sovereignty and deterministic retrieval speeds while strictly adhering to the resource constraints of standard consumer hardware (e.g., 16GB RAM). Ultimately, this project demonstrates that enterprise-grade, natural language document retrieval can be achieved locally, transforming chaotic file systems into highly accessible knowledge bases without sacrificing performance or privacy.

Area of Study: Desktop App Development, NLP, Semantic Search, File Indexing

Keywords: semantic search, vector embeddings, desktop application, information retrieval, Rust programming, Tauri framework, local processing, document search, natural language processing, privacy-preserving

TABLE OF CONTENTS

TITLE PAGE	i
ABSTRACT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	vi
CHAPTER 1 PROJECT BACKGROUND	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Contributions	4
CHAPTER 2 LITERATURE REVIEW	2
2.1 Previous Works on File Retrieval	5
2.2 Previous Works on Sentence Embeddings Algorithm	9
CHAPTER 3 PROJECT SCOPE AND OBJECTIVES	11
3.1 Project Scope	11
3.2 High-Level Architectural Diagram	13
3.3 Project Objective	15
CHAPTER 4 METHODS/TECHNOLOGIES INVOLVED	16
4.1 Methodology: Iterative and Incremental Development (IID)	16
4.2 Gantt Chart: A Blueprint for IID Approach	17
4.3 Tools	20
REFERENCES	22

LIST OF FIGURES

Figure Number	Title	Page
Figure 2.1	Mango Finder	8
Figure 2.2	Renamer.ai	10
Figure 2.3	File Brain	13
Figure 2.4	Bulk Rename Utility	14

LIST OF TABLES

Table Number	Title	Page
Table 3.1	Specifications of laptop	17

LIST OF ABBREVIATIONS

<i>RAG</i>	Retrieval-Augmented Generation
<i>NPU</i>	Neural Processing Units
<i>LLM</i>	Large Language Model
<i>NLP</i>	Natural Language Processing
<i>GUI</i>	Graphical User Interface
<i>ONNX</i>	Open Network Neural Exchange

CHAPTER 1: Project Background

1.1 Introduction

The rapid proliferation of digital documents has turned local file systems into fragmented landscapes. Users frequently accumulate thousands of files and reference materials across various directories, leading to an overwhelming state of digital clutter [1]. While native operating system search features are mature and efficient at exact keyword matching, they fundamentally fail to align with human cognitive recall. People rarely memorize rigid file naming conventions or exact string literals. Instead, they remember the conceptual purpose of a project, or the main idea of a document. When a user recalls the underlying context but forgets the specific phrasing, traditional search engines yield empty results. This forces users into a frustrating, time-consuming process of manually traversing deeply nested directories, breaking their operational flow and distract themselves from actual work, for the sake of retrieving their own data.

While mainstream artificial intelligence offers powerful contextual understanding, it has forced users to compromise their privacy and data security, which is to rely on cloud-based Retrieval-Augmented Generation (RAG) pipelines that expose sensitive personal files to risks. Although the tech industry is now shifting toward local AI processing, these features are heavily hardware-gated, requiring expensive Neural Processing Units (NPUs) and imposing immense system overhead. To prove this, Microsoft's official developer guidance for integration of Copilot with Windows 11 explicitly establishes a strict hardware barrier for native local AI processing [3]. To access these OS-level AI capabilities, users are mandated to own a completely new class of devices equipped with NPUs. QorFinder aims to be a lean desktop application engineered to bypass these bloats. By executing lightweight, transformer-based embedding models directly on standard consumer hardware, it delivers fast, private, and zero-cost semantic file search which guarantees absolute data sovereignty without the computational bloat.

1.2 Problem Statement

1.2.1 The Semantic Gap in File Search

The core limitation of traditional lexical file search app is the semantic gap between human intent and computer logic. Users often cannot recall the specific keyword, but only recall ideas, relationships, and summaries while standard local search tools enforce strict lexical logic, requiring exact character matches [4]. This cognitive mismatch results in key information being effectively lost within a user's own hard drive. The inability to efficiently locate these files carries a massive productivity cost. Studies [2] estimate that knowledge workers spend around 20% of their time simply searching for information, representing a substantial economic waste and causing deep frustration due to constantly interrupted workflows.

1.2.2 Limitations of Current Approach

Users attempting to deal with this digital clutter currently face inadequate options. Manual approaches, such as batch-renaming tools or strict folder hierarchies, require continuous configuration and are rigid, often breaking when a user's workflow evolves.

Alternatively, using RAG search tools presents a false compromise. RAG relies on Large Language Model (LLM) to generate responds based on given source, which is computationally intensive. By using RAG, users can gain semantic search functionality, but at what cost? Although the cloud server provides almost infinite capacity, consumer and edge devices are known for their inadequate memory and processing capabilities, thus posing a serious challenge to accuracy when it comes to deploying the billion-parameter model [5]. This is a dilemma since users must either surrender their data sovereignty to cloud service providers or purchase costly hardware to run models locally. To preserve privacy, users need to purchase high-end hardware, which the return does not justify the investment. It is creating financial problems while attempting to solve the original one. Furthermore, as highlighted in [7], LLMs are still prone to hallucinations, which will generate content that is factually incorrect correspond to the source, which is due to underlying architectural limitations and training data inconsistencies.

1.2.3 Stakeholders

The lack of a secure, local semantic search tool severely impacts a broad range of computer users. The primary stakeholders include information-heavy knowledge workers, such as students, academic researchers, writers, and software developers. They must manage massive amount of documentation where the conceptual meaning of a file is far more important than its specific keyword. Furthermore, this problem affects privacy-conscious professionals, including legal practitioners and small business owners. These individuals either not willing to use cloud AI services, or being budget tight, which causes them to entirely cut off from modern contextual search capabilities.

1.3 Motivation

The motivation behind QorFinder originated from the author's frustration. He once gets into a situation that, all existing backup files' name had been modified to meaningless numbers due to a wrongly executed script. It is painful to manually check each file's content, then rename them for future retrieval. Part of the files are lost forever, since there was no time to review files of 200GB in detail. He could not afford purchasing high-end hardware to host local LLM, also refuse cloud solutions due to privacy considerations and budget constraints.

The idea of this project is rooted in the necessity for local-first AI tools that prioritise user privacy without sacrificing execution performance. As the volume of personal data grows, there is a clear demand for a "set-and-forget" indexing service that empowers users to interact with their data intuitively, ensuring information is accessible regardless of how it was named or where it was stored. This project is further motivated by the technical challenge of memory safety and efficiency to the field of Information Retrieval. The improvement of efficient embedding techniques now makes it feasible to bring enterprise-grade search capabilities to standard laptops. By building QorFinder, the goal is to demonstrate that advanced semantic search can be achieved on-device, providing a seamless and secure user experience that keeps sensitive data under the user's absolute control.

1.4 Contributions

The contribution of this project lies in the integration of **high-performance software modules** and **optimized algorithms**. It is normal that this project invented none of the modules, however the integration itself yields novelty. This work will demonstrate that enterprise-grade file retrieval can be achieved on standard consumer hardware without external dependencies. The components integrated are as follows:

The most crucial component is a lightweight transformer model, which is designed for the highly efficient generation of vector embeddings. This shall prove that Natural Language Processing (NLP) can operate locally without sacrificing performance.

Secondly, the implementation of a performant database, Qdrant, is tailored specifically for minimal memory consumption during active operation. This creates a unified storage solution that handles both traditional file metadata and complex vector embeddings without straining local system resources.

Thirdly, this project will utilise Rust programming language to handle calculation intensive operations. This could minimise computing overhead and leverages strict memory safety to prevent runtime bugs, guaranteeing a stable, responsive application even when processing massive local directories. This project also uses Tauri framework, which allows using web technologies to develop frontend Graphical User Interface (GUI), maintaining backend as Rust.

Ultimately, this architecture resolves the contradiction between data privacy and hardware cost. By intentionally trading heavy generative functionalities for optimized retrieval, the system is fine-tuned to hit the exact sweet spot between search accuracy, execution speed, and resource efficiency.

CHAPTER 2: Literature Review

2.1 Previous Works on File Retrieval

The landscape of local file retrieval and organization spans from modern AI-driven semantic engines to foundational lexical indexers, each utilizing distinct architectures to address information discovery. At the forefront of this evolution are semantic file search tools such as FileBrain (Figure 2.3) and MangoFinder (Figure 2.1), which attempt to resolve the traditional "vocabulary mismatch" problem by processing the underlying intent of a user's query [9, 10]. For example, MangoFinder acts as a desktop application which is based on Tauri, where its backend is written in Rust to achieve memory safety, while its frontend is developed using TypeScript and JavaScript to keep its binary size small. To run sentence transformers to generate high dimensional vectors from natural language queries on local devices, these applications rely on ONNX Runtime [9]. These vector embeddings are managed through SQLite database with vector extensions and are used to execute complex hybrid queries involving semantic similarity with metadata filtering. MangoFinder also offers multi-device searching provided the devices are connected under the same local network. In contrast, File Brain acts similar as MangoFinder, but the difference lies in the programming languages used. MangoFinder uses Rust, whereas FileBrain uses Python, hence being slower in indexing compared to MangoFinder. However, both are cross-platform [9, 10].

Before retrieval systems can operate effectively on legacy or chaotic data, the underlying files often require aggressive standardization, a task best suited for Batch File Rename tools. While developers and power users can easily write terminal scripts (using Bash or PowerShell) to execute mathematically predictable, low-risk string replacements, graphical user interface (GUI) applications like Bulk Rename Utility (BRU, Figure 2.4) and Renamer.ai excel in complex, high-stakes environments [11, 12]. GUI tools provide real-time, side-by-side visual previews of proposed changes, preventing catastrophic batch errors that a poorly formatted regular expression might cause in a terminal. Furthermore, BRU can natively hook into deep file metadata—such as extracting EXIF creation dates from photographs or ID3 tags from audio files—bypassing the tedious command-chaining required in command-line environments.

Modernizing this workflow, Renamer.ai (Figure 2.2) utilizes local machine learning to read the actual content of documents, allowing the software to automatically rename a file based on contextual intent (e.g., extracting vendor names and totals from a scanned invoice) rather than relying on rigid rules [12].

Despite the advancements in semantic understanding and automated organization, lexical search engines stay the foundation of file searching because of unrivaled computational efficiency. Voidtools' "Everything" is considered the most efficient engine for searching on global scale on Windows platform [13]. The program can process searches instantaneously, even involving millions of files since it totally avoids typical, iterative traversal of directories. Rather than that, Everything uses direct reading of Master File Table of NTFS file system, providing instant map of the host drive. Ripgrep becomes the speed limit in searches for exact string matches in raw text [14]. Written entirely in Rust, ripgrep combines a highly optimized regular expression engine with the Aho-Corasick algorithm, allowing it to search multiple strings simultaneously in linear time. Additionally, it employs Single Instruction, Multiple Data (SIMD) optimizations via the "Teddy" algorithm, which is borrowed from Intel's Hyperscan project—to evaluate large chunks of text in parallel at the CPU register level, offering unmatched performance for developer-centric code and log searches [14].

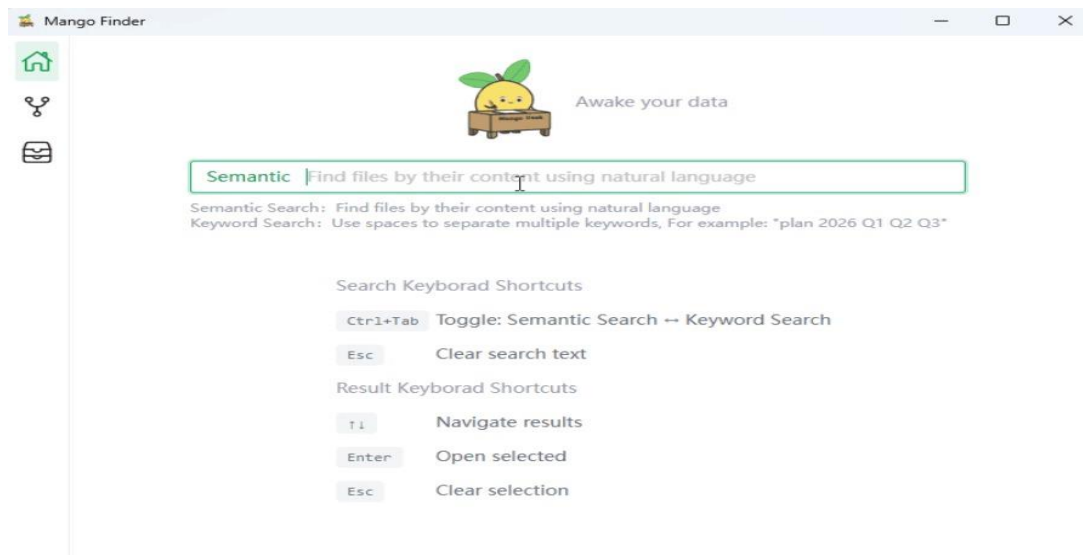


Figure 2.1: Mango Finder

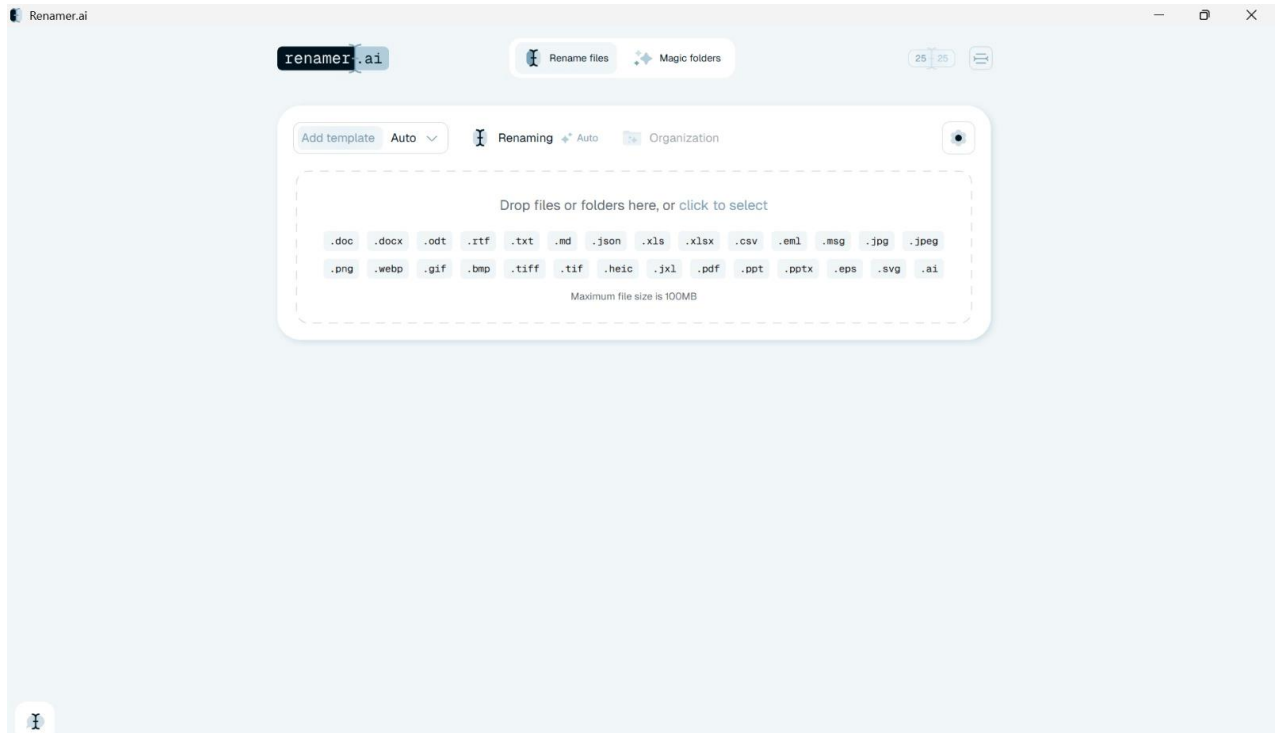


Figure 2.2: Renamer.ai

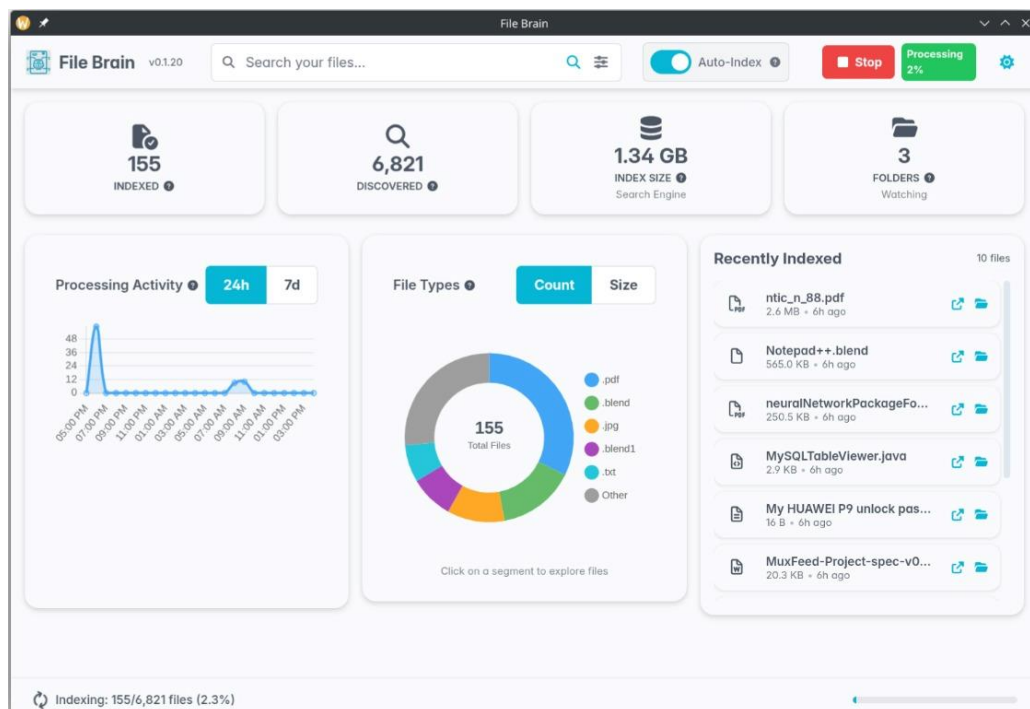


Figure 2.3: File Brain

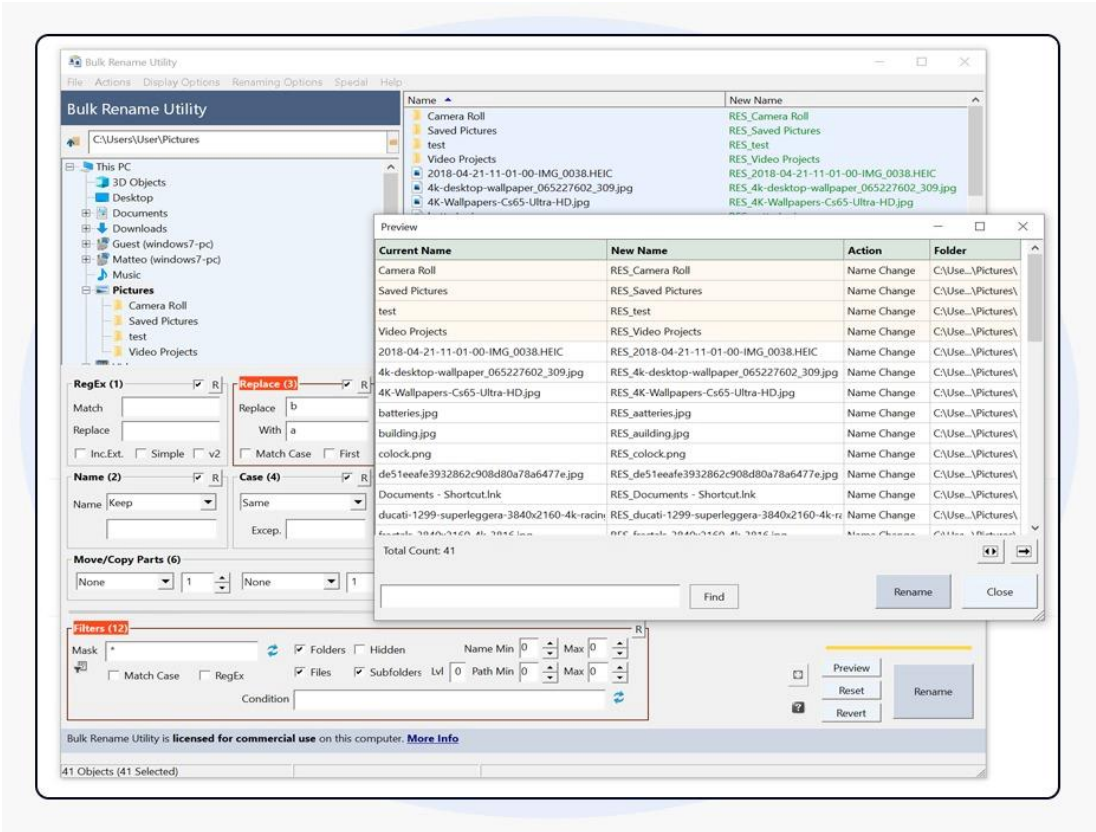


Figure 2.4: Bulk Rename Utility

2.2 Previous Works on Sentence Embeddings Algorithm

The transition from traditional keyword search to semantic retrieval is powered by the famous transformer architecture, which is capable to capture deep, context-aware relationships between tokens [6]. While leveraging self-attention for parallel token processing, it serves as the basis for Bidirectional Encoder Representations from Transformers (BERT). Through bidirectional encoding via masked language modeling, BERT achieves high semantic accuracy, with Spearman correlations of 0.75 to 0.85 on the Semantic Textual Similarity Benchmark (STS-B) when evaluating sentence pairs jointly [15]. While BERT is architecturally designed to generate contextualized vectors for individual tokens, it lacks a native fixed-size representation for entire sequences. To address this, a mathematical pooling operation, such as mean or max pooling is applied to aggregate these token-level embeddings into a single, comprehensive sentence embedding [24]. However, to maintain high similarity accuracy, BERT unfortunately requires joint processing for pairs, which severely limits scalability.

A Siamese Network is defined as an architecture utilizing twin subnetworks with shared weights to map disparate inputs into a unified vector space. Sentence-BERT (SBERT) adapts this framework by integrating mean pooling to aggregate token-level data into fixed-size embeddings suitable for cosine similarity. This architectural shift maintains high semantic accuracy—achieving an average Spearman correlation of 0.80–0.85 on STS tasks—while dramatically improving scalability. Specifically, this approach achieving comparable STS performance (average Spearman 0.80-0.85) while reducing search time over 10,000 sentences to 5 seconds from BERT's 65 hours [24]. SBERT-WK is one such variant which uses a geometrically unified method for aggregating information from all Transformer layers, considering not only syntactic but also semantic components which usually get omitted from single layer pooling methods. This results in 10-15% performance gains on STS task compared to baseline pooling [16]. Notably, Token Attention Sentence-BERT (TA-SBERT) makes use of a token-level attention mechanism to assign weights based on their significance, thus ensuring that only important tokens make greater contributions to the overall vector, without fine-tuning overhead [18]. While these architectural adaptations consistently yield superior embedding quality on STS-B, they introduce a critical design trade-off

between semantic precision and inference speed. Specifically, geometric layer fusion provides a more robust 'semantic fingerprint' by utilizing the model's entire internal structure, yet it incurs a measurable computational penalty. In contrast, simple mean pooling remains the most efficient baseline for high-throughput local environments, even if it lacks the nuanced weighted representation provided by more complex attention-based or geometric methods.

Bi-Encoders operate by independently encoding queries and documents into a shared vector space. As documents can be pre-processed and indexed, similarity searches are incredibly fast, typically taking only 100 to 200 microseconds per sentence. Models like LaBSE and MUSE demonstrate this scalability, achieving 83.7% bi-text retrieval accuracy across 112 languages with significantly less training data than baselines [19, 20]. Yet, because the query and document never interact during the encoding phase, Bi-Encoders lack the nuanced ranking capabilities required for complex semantic matching. Conversely, Cross-Encoders process the query and document jointly as a single input. This full-interaction approach allows the model's self-attention mechanism to perform deep, word-for-word comparisons, resulting in vastly superior, nuanced ranking that yields statistically significant improvements over traditional algorithms like BM25 [17]. However, this joint processing creates severe latency, making Cross-Encoders entirely unsuitable for searching large corpora, as scores cannot be pre-computed. To reconcile this dichotomy, researchers employ knowledge distillation to bridge the gap between speed and precision. In this paradigm, a highly accurate but slow "teacher" model (the Cross-Encoder) transfers its complex semantic signals to a faster "student" model (the Bi-Encoder). Approaches like ERNIE-Search utilize "on-the-fly" distillation to achieve state-of-the-art dense passage retrieval, capturing the deep interaction benefits of a Cross-Encoder while maintaining the fast, independent indexing of a Bi-Encoder [21, 22]. Furthermore, listwise distillation with soft labels teaches the Bi-Encoder how to rank entire lists of documents accurately, resulting in nearly a 19.66% improvement in top-tier retrieval metrics on complex datasets like SQuAD [23]. Ultimately, the synthesis of quantization techniques and knowledge distillation ensures that modern semantic search engines can deliver unparalleled accuracy without sacrificing the speed and efficiency required for localized deployment.

CHAPTER 3: Project Scope and Objectives

3.1 Project Scope

The scope of the project includes the development of a full package of a semantic search desktop application, QorFinder that works only in Windows 10/11. QorFinder will be built with Rust, powered by Tauri, which enables the web tech stack to build the frontend. QorFinder offers extremely fast and accurate semantic searching of general files, under a limitation of 16GB of RAM. The system will be designed in an entirely dedicated way for offline indexing of text documents only (.txt, .pdf, .docx, .md) via Qdrant and sentence transformers that will guarantee ultimate security and immediate access. To remain aligned with the stakeholders, it deliberately exclude features such as cloud synchronisation and web hosting, so that privacy and security could be preserved. QorFinder is designed to handle only text documents, since stakeholders are managing massive amount of documentation.

3.2 High-Level Architectural Diagram

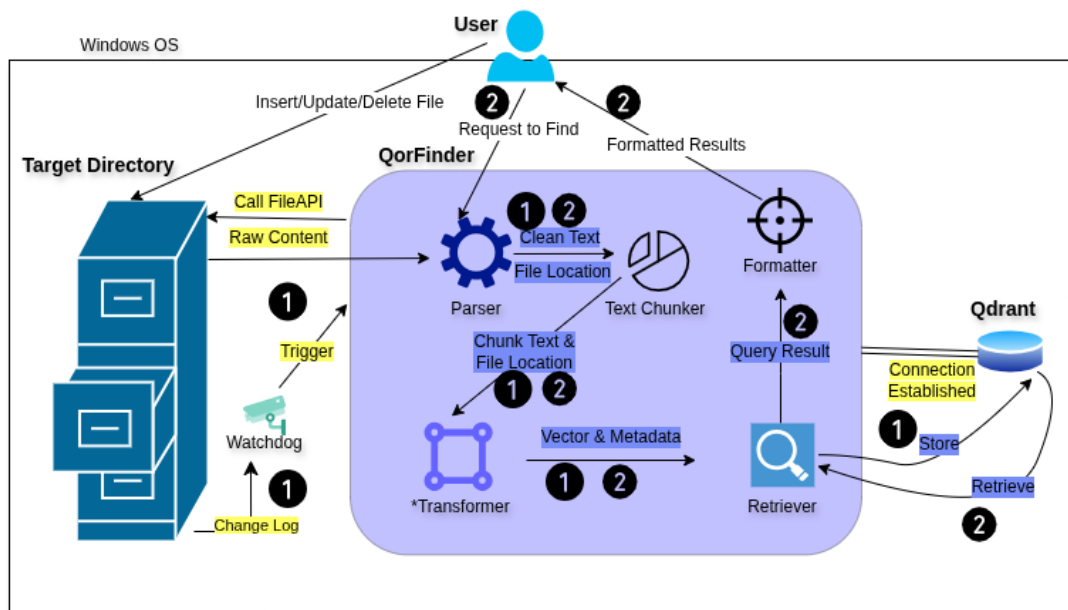


Figure 3.1: Data Flow Diagram of QorFinder

Referring to Figure 3.1, the explanation of the intended workflow is shown below.

1 Initialisation/Background Daemon: QorFinder track target directory for ongoing file changes using watchdog. If there are changes, then watchdog sends a trigger to backend. Backend immediately call the NTFS API to retrieve file content. The parser will clean the noise that exists within the raw content. Then it passes the cleaned text and file location metadata to the text chunker. The text chunker will then chunk text to meaningful pieces. It passes chunked text and file location metadata to the chosen transformer model. The model will generate dense vector based on the chunked text. It passes the sentence embeddings (which is the dense vector) along with file location metadata to the retriever. Retriever will store these data into the Qdrant instance. This procedure ensures that data in target directory are all indexed, which laid a foundation for user to perform semantic search.

2 User Perform Semantic Search: User wants to find a file which contains some meaning. He/she gives a prompt which serves as an input to QorFinder. The parser accepts these input and clear unnecessary symbols that may exist, then pass it to text chunker. Text chunker process the prompt if it is too lengthy, into standard size of chunk, then pass it to the transformer model. The model converts it to dense vector embeddings, which will then be transferred to retriever. Retriever will retrieve result based on the distance of the query's sentence embeddings and the vectors stored. The list of retrieved vectors and metadata is then organised by formatter, then presented to the user.

Figure 3.2 provides a clean summary of components of QorFinder.

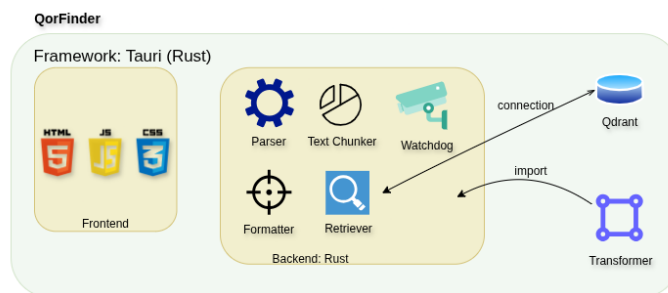


Figure 3.2: Components of QorFinder

3.3 Project Objectives

Main Objective

To design, engineer, and deploy an efficient and private semantic file search desktop application that supports the fast and effective searching of English documents via natural language inputs using only local hardware without cloud services.

Sub Objective

1. To design and implement a loosely-coupled architecture using the Rust language, Tauri and Qdrant to achieve efficient indexing of English-language files under 16GB RAM limits.
2. To iterate and fine-tune the integration of sentence transformer models and semantic retrieval logic to maximize accuracy while maintaining low-latency CPU inference.
3. To test results through performance benchmarking and security audits to verify deterministic retrieval speeds, data privacy, and the integrity of incremental file system updates.

CHAPTER 4: Methods/Technologies Involved

4.1 Methodology: Iterative and Incremental Development (IID)

QorFinder applies Iterative and Incremental Development (IID) in its development methodology. The IID approach is a software development process whereby the project is divided into a series of iterations in which an increment of the product is produced, whereas the Waterfall technique follows a linear sequential flow of all development stages without delivering anything until the entire system development is complete [8].

Unlike Agile, the IID approach can be described as mini-Waterfall approach because the analysis, design, and coding tasks are completed sequentially during each iteration cycle in a smaller scale [8]. Conversely, the Agile process requires that the whole range of activities such as planning, coding, and testing should be performed collaboratively in a single iteration or sprint [8]. Figure 4.1 shows the workflow of Iterative and Incremental Development.

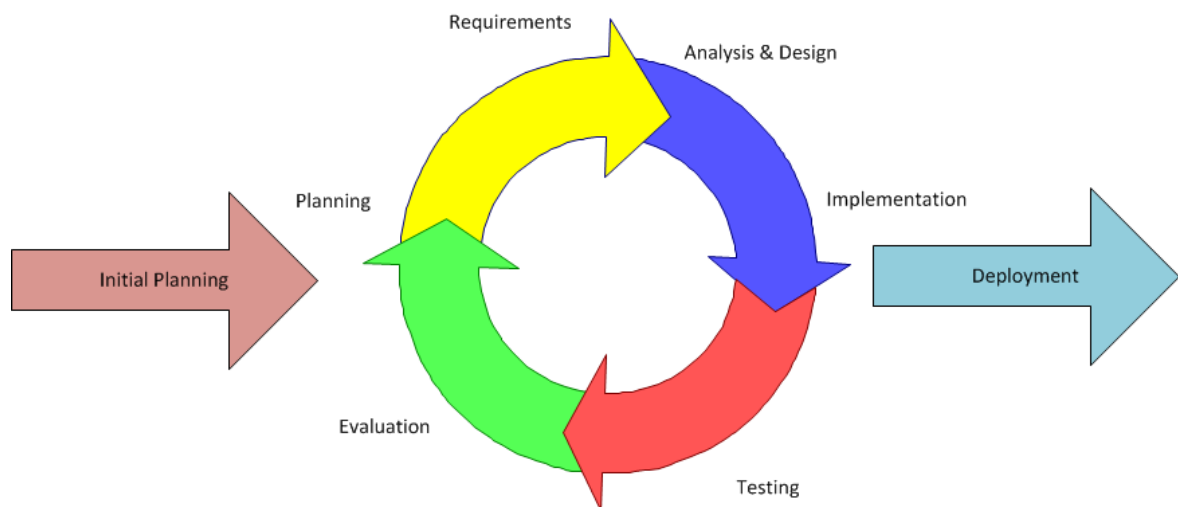


Figure 4.1: Iterative and Incremental Development [8]

4.2 Gantt Chart: A Blueprint for IID Approach

4.2.1 Seven Increments Throughout FYP 1 and 2

There are two gantt charts, which the Figure 4.2 is about 3 increments (FYP1), and Figure 4.3 is about 4 increments (FYP2). Each increment has a deliverable, and having a mini-Waterfall approach, means having planning, requirements gathering, analysis & design, implementation, testing and evaluation as shown in [Figure].

For **Increment 1**, Rust development environment will be established and Qdrant instance is configured locally. All dependencies are verified to compile on Windows 10/11. It is also needed to define the project structure for loose coupling between Tauri frontend and Rust backend.

For **Increment 2**, evaluate multiple quantized sentence transformer models (BGE-M3, all-MiniLM-L6-v2, etc) by benchmarking semantic accuracy against a test corpus of several English documents. The inference latency on CPU and RAM consumption shall be measured to select the optimal model for 16GB RAM hardware.

For **Increment 3**, develop a CLI program implementing the entire document retrieval process by (1) extracting texts from the file sources, (2) encoding the texts using the chosen model, (3) saving the embeddings to Qdrant, and (4) answering natural language queries with semantically similar documents. The validation of the functionality involves successful document retrieval for at least 70% of the testing queries.

For **Increment 4**, apply quantization (Int8/Float16) and fine tuning to the chosen model. It will be measured the trade-off between accuracy loss (should be above 75%) and performance gains. This ensures QorFinder runs smoothly on resource-constrained Windows laptops.

For **Increment 5**, use the notify library in Rust to monitor files. The watch will recognize changes such as creation, modification, and deletion of files in directories chosen by users. QorFinder will then index only changed files incrementally rather than re-indexing the entire corpus, which is critical for performance with large document collections.

For **Increment 6**, perform rigorous system integration and debugging. Edge cases will be considered, such as scenarios like large PDFs (>50MB), encrypted or corrupted files,

REFERENCES

permission errors, database lock conflicts, and concurrent read/write operations. This involves unit testing and integration testing of the entire process.

For **Increment 7**, develop the Tauri application with a frontend based on HTML, CSS, and JavaScript. For example, integration with the backend search engine would be accomplished using the Tauri IPC Bridge. The final step is to package all pieces into a single Windows .exe executable without any dependencies.

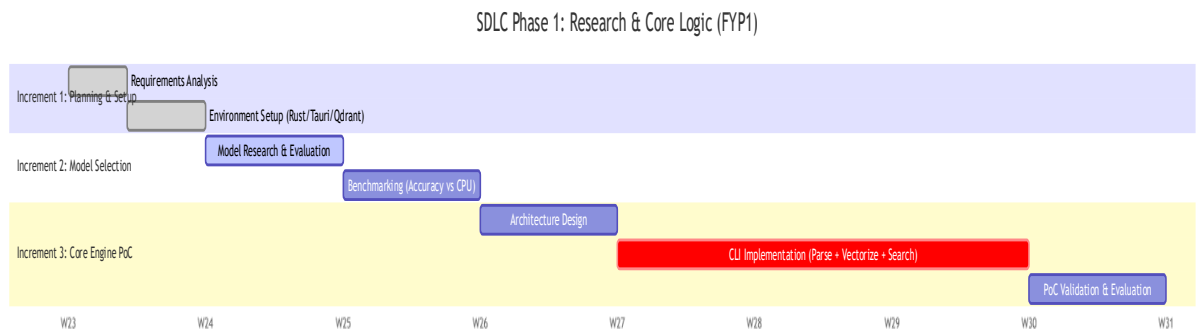


Figure 4.2: Gantt Chart for FYP1

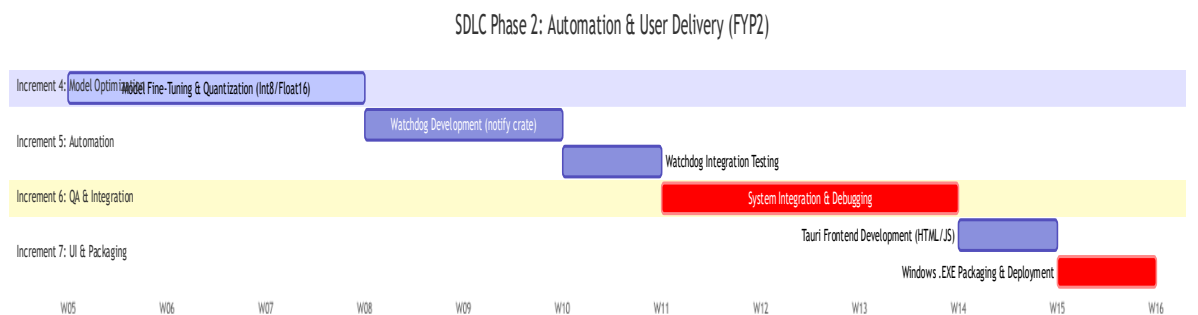


Figure 4.3: Gantt Chart for FYP2

4.2.2 Achieving the Objectives

The **first sub-objective will be mainly met** by Increments 1 and 3, with the validation occurring in Increment 6, wherein a loosely-coupled architecture of Rust, Tauri, and Qdrant will work efficiently within the limits of 16GB RAM.

The **second sub-objective will mainly be met** by Increments 2, 3, and 4, through iterations in all the three increments for fine-tuning sentence transformers and semantic retrieval algorithms to attain maximum precision while ensuring CPU inference latency remains low through processing.

The **third sub-objective is met** by Increments 3, 5, and 6, whereby the security process occurs during all the phases with regards to achieving deterministic retrieval speeds, complete privacy through offline-only retrieval, and incremental file system updates.

Lastly, **Increment 7 combines all deliverables** in the first six increments: backend, optimization of models, watchdog mechanism, and quality assured core, into one installer package for Windows .exe format **to achieve the main objective** of a desktop application for searching semantic files privately and offline in English documents.

All Increments has been summarized in Table 4.1 for clarity.

Table 4.1: Summary of Seven Increments of QorFinder Development

Increment	Phase	Deliverable	Validation Criteria
1	Setup	Rust/Tauri/Qdrant environment	Compiler runs, Qdrant instance responds
2	Model Selection	Benchmark report of some competitive sentence transformers	Model with best accuracy or CPU tradeoff identified
3	PoC	CLI semantic search tool	Can parse text, vectorize, and retrieve results

REFERENCES

4	Model Optimization	Quantized model and fine tuning	RAM usage reduced, inference speed improved
5	Automation	File system watchdog	Auto-detects add/modify/delete events
6	Quality Assurance	Integrated and debugged system	Handles PDFs, .docx, .md; resolves edge cases
7	UI & Packaging	Windows .EXE installer	Full GUI application ready for deployment

Table: A Summary of Project Increments

4.3 Tools

4.3.1 Hardware

A laptop is selected to evaluate and deploy QorFinder. This laptop ensured the compatibility, functionality, and performance in real-world scenarios. The laptop's hardware specifications are presented in Table 4.2.

Table 4.1 Specifications of a 16GB RAM Laptop

Description	Specifications
Model	Lenovo ThinkPad E14 Gen 3
Processor	AMD Ryzen 5 5500U (6 Cores, 12 Threads, up to 4.0 GHz)
Operating System	Windows 10 Pro
Graphic	AMD Radeon Graphics (Integrated)
Memory	16GB DDR4-3200Mhz
Storage	512 GB

4.3.2 Software

QorFinder takes a local-first approach, where the lightweight web interface is connected to the high-performance backend through the Inter-Process Communication bridge provided by Tauri. To keep the resource requirements low but development practical, the user interface has been implemented using plain Vanilla JavaScript along with HTML and CSS. The design guarantees that the page loads instantaneously while offering a native experience on both Windows 10 and 11. With the help of the Tauri framework, the executable file remains extremely light, consuming no additional memory during execution. The interface shall remain very simple for easy indexing of directories, performing searches, and filtering of results.

The backend is built using Rust, selected for its inherent memory safety guarantees and robust fearless concurrency support. It delivers exceptional execution speed by eliminating the latency associated with garbage collection. As all critical functionality operates entirely offline, the backend utilizes the walkdir and notify crates for efficient recursive directory traversal and real-time file change detection. For AI inference, especially using the sentence transformer model, the backend leverages the ONNX Runtime engine. It allows QorFinder to quickly compute semantic embeddings from the text of documents using the local processing power. This allows users full ownership of their data without relying on external cloud systems, at the same time ensures functionality.

For data management, the application integrates Qdrant operating strictly in an embedded, local mode. Serving as a unified storage engine without reliance on background server daemons, Qdrant is optimized to handle both high-dimensional dense vectors and structured document metadata (such as file paths, extensions, and timestamps). This enables complex hybrid search queries—combining semantic similarity with precise metadata filtering—while maintaining minimal I/O overhead. Together, this specialized stack (Rust, Tauri, Qdrant, and ONNX) guarantees robust performance well within standard hardware constraints, which is assumed a 16GB RAM system. The application's development lifecycle is supported by version control tools (git) and systematic testing via cargo test.

REFERENCES

- [1] P. Englefield and R. Beale, "Managing personal information: tactics, strategies, and outcomes," *Interacting with Computers*, Dec. 2025, doi: <https://doi.org/10.1093/iwc/iwaf054>.
- [2] S. Williams, "Australian workers waste time searching for information," *IT Brief Australia*, Mar. 16, 2025. [Online]. Available: <https://itbrief.com.au/story/australian-workers-waste-time-searching-for-information>
- [3] Microsoft Corporation, "Develop AI applications for Copilot+ PCs," *Microsoft Learn*, Nov. 17, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/ai/npu-devices/>
- [4] P. Nigam *et al.*, "Semantic Product Search," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Jul. 2019, doi: <https://doi.org/10.1145/3292500.3330759>.
- [5] X. Wang, Z. Xu, and X. Sui, "Intelligent data analysis in edge computing with large language models: applications, challenges, and future directions," *Frontiers in Computer Science*, vol. 7, May 2025, doi: <https://doi.org/10.3389/fcomp.2025.1538277>.
- [6] A. Vaswani *et al.*, "Attention Is All You Need," *arXiv.org*, 2017. <https://arxiv.org/abs/1706.03762>
- [7] L. Huang *et al.*, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM transactions on office information systems*, vol. 43, no. 2, Nov. 2024, doi: <https://doi.org/10.1145/3703155>.
- [8] "Hybrid Agile Methodology: Spiral, V-Model & More," *Inflectra.com*, 2024. <https://www.inflectra.com/Solutions/Methodologies/Hybrid.aspx>
- [9] Tauri Apps, "Awesome Tauri: MangoFinder," *GitHub*, 2026. [Online]. Available: <https://github.com/tauri-apps/awesome-tauri>.

REFERENCES

- [10] Hamza Abbad, "File Brain," *File Brain*, 2024. <https://file-brain.com> (accessed Apr. 27, 2026).
- [11] TGRMN Software, "Bulk Rename Utility," *Official Website*, 2024. [Online]. Available: <https://www.bulkrenameutility.co.uk/>.
- [12] Renamer.ai, "Batch File Renamer," *Official Website*, 2026. [Online]. Available: <https://renamer.ai/>.
- [13] Voidtools, "Everything Search Engine," *Official Website*, 2025. [Online]. Available: <https://www.voidtools.com/>.
- [14] A. Gallant, "ripgrep: recursively searches directories for a regex pattern," *GitHub*, 2026. [Online]. Available: <https://github.com/BurntSushi/ripgrep>.
- [15] Y. Yang et al., "Multilingual Universal Sentence Encoder for Semantic Retrieval," Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-demos.12. Available: <https://doi.org/10.18653/v1/2020.acl-demos.12>
- [16] B. Wang and C.-C. J. Kuo, "SBERT-WK: A Sentence Embedding Method by Dissecting BERT-Based Word Models," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28. Institute of Electrical and Electronics Engineers (IEEE), pp. 2146–2157, 2020. doi: 10.1109/taslp.2020.3008390. Available: <https://doi.org/10.1109/taslp.2020.3008390>
- [17] K. Vowinckel and V. D. Hähnke, "SEARCHFORMER: Semantic patent embeddings by siamese transformers for prior art search," *World Patent Information*, vol. 73. Elsevier BV, p. 102192, Jun. 2023. doi: 10.1016/j.wpi.2023.102192. Available: <https://doi.org/10.1016/j.wpi.2023.102192>
- [18] J. Seo, S. Lee, L. Liu, and W. Choi, "TA-SBERT: Token Attention Sentence-BERT for Improving Sentence Representation," *IEEE Access*, vol. 10. Institute of Electrical and Electronics Engineers (IEEE), pp. 39119–39128, 2022. doi: 10.1109/access.2022.3164769. Available: <https://doi.org/10.1109/access.2022.3164769>

REFERENCES

- [19] Y. Yang et al., "Multilingual Universal Sentence Encoder for Semantic Retrieval," arXiv (Cornell University), 2019, doi: 10.48550/arxiv.1907.04307. Available: <https://doi.org/10.48550/arxiv.1907.04307>
- [20] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, "Language-agnostic BERT Sentence Embedding," Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.62. Available: <https://doi.org/10.18653/v1/2022.acl-long.62>
- [21] Y. Lu et al., "ERNIE-Search: Bridging Cross-Encoder with Dual-Encoder via Self On-the-fly Distillation for Dense Passage Retrieval," arXiv (Cornell University), 2022, doi: 10.48550/arxiv.2205.09153. Available: <https://doi.org/10.48550/arxiv.2205.09153>
- [22] H. Ren, L. Shou, N. Wu, M. Gong, and D. Jiang, "Empowering Dual-Encoder with Query Generator for Cross-Lingual Dense Retrieval," Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.203. Available: <https://doi.org/10.18653/v1/2022.emnlp-main.203>
- [23] S. B. Desai, "Distilling Cross-Encoder Signals into Bi-Encoders for Domain Retrieval," Frontiers in Emerging Artificial Intelligence and Machine Learning, vol. 02, no. 10. Emerging Innovations Society: Science, Technology, and Social Studies, pp. 45–54, Oct. 30, 2025. doi: 10.64917/feaiml/volume02issue10-04. Available: <https://doi.org/10.64917/feaiml/volume02issue10-04>
- [24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," TUBilio (Technical University of Darmstadt), 2019, doi: 10.48550/arxiv.1908.10084. Available: <https://doi.org/10.48550/arxiv.1908.10084>

IIPSPW Report Marking Sheet

Mark Allocation	Key Assessment	Marks
10.0%	Project Background - What is the problem domain of this project? - Who has the problem and need a solution? - Why is the problem so important?	
10.0%	Literature Review - What have other researchers/developers done to resolve the problem? - What are the strengths of their solutions? - What are the weaknesses/limitations of their solutions? - How these weaknesses/limitations can be resolved?	
10.0%	Project Scope and Objectives - Is the title clear and specific enough? - What is the proposed solution of this project to tackle the above problem/limitations? - What is the scope? - What is the main objective, and how this objective can be divided into sub-objectives of the project? - What is the innovation/contribution of the project?	
5.0%	Methods/Technologies Involved - What are the proposed methods/technologies involved in the solutions? - How to justify the objective/sub-objectives can be achieved with these methods/technologies	
5.0%	Report Format / Grammar / References - Are the references adequate? - Is the report free from grammatical and writing errors?	
40.0%	Total	